

**PROJECT REPORT OF CO-OP PROJECT at INDUSTRY
(MODULE-II)
ON**

AI Adoption Analytics – Developer AI Proficiency Measurement System

**BACHELOR OF ENGINEERING
In
COMPUTER SCIENCE AND ENGINEERING (ARTIFICIAL
INTELLIGENCE)**

Submitted by:

Sambhav Singh (2210993840)

Supervised By:

Dr. Tanvi Sood
Assistant Professor
Chitkara University



**CHITKARA UNIVERSITY INSTITUTE OF ENGINEERING & TECHNOLOGY
CHITKARA UNIVERSITY, RAJPURA**

Jun 2026

CONTENTS

S.No.	Title	Page No.
1	Declaration	3
2	Acknowledgement	4
3	Abstract	5
4	Introduction to the Problem Statement	6-8
5	Research Methodology	9-12
6	System Flow Chart	13-14
1	Tools and Technologies Used	15-16
8	Solution Implementation	17-18
9	Major Findings/Outcomes/Output/Results	19-21
10	Conclusion and Future Scope	22-23
11	References	24
12	Appendices	25

DECLARATION

I hereby declare that the project work titled, "AI Adoption Analytics – Developer AI Proficiency Measurement System" submitted as part of Bachelor's degree in CSE (ARTIFICIAL INTELLIGENCE), at Chitkara University, Punjab, is an authentic record of my own work carried out under the supervision of **Dr. Tanvi Sood** during my industry co-op at R Systems International.

Signature:

ACKNOWLEDGEMENT

With immense pleasure I, Sambhav Singh, am presenting the "AI Adoption Analytics" project report as part of the curriculum of 'BE-CSE (AI)'.

I would like to express my sincere thanks to Dr. Tanvi Sood, for valuable guidance and support in completing my project.

I would also like to express my gratitude towards the faculty and leadership at Chitkara University for giving me the opportunity to undertake this industry co-op project at R Systems International. Without their support and suggestions, this project would not have been completed.

I extend my appreciation to the mentors and engineering team at R Systems International for providing access to real-world AI adoption data, development infrastructure, and feedback throughout the implementation of the pipeline.

Signature.....

Name: Sambhav Singh

Roll No: 2210993840

Abstract

AI Adoption Analytics is a comprehensive developer analytics system developed to convert raw AI-tool telemetry, uploaded evidence artifacts, and usage patterns into actionable proficiency intelligence. The project integrates data engineering, large language models, vector similarity search, and rule-based auditing to uncover how engineers actually use AI assistants such as Cursor and GitHub Copilot.

The system automates key processes such as API-based data collection, MySQL storage, evidence summarization, profile generation, competency scoring, and 7-day usage-pattern auditing to ensure accuracy and transparency. Using advanced analytical models and embedding-based matching against a 43-item competency framework, AI Adoption Analytics evaluates developer proficiency, detects anomalous usage behaviour, and produces meaningful interpretations that support strategic workforce planning.

Interactive dashboards through a Streamlit web interface and batch processing for entire engineering rosters enhance organizational understanding and make complex adoption metrics easy to interpret. By combining modern AI techniques with real-world enterprise applications, AI Adoption Analytics aims to empower organizations with deeper insights, improved talent development, and evidence-based decision-making around AI tool adoption.

CHAPTER 1 — INTRODUCTION TO THE PROBLEM STATEMENT

1.1 Background and Context

In today's software-driven organizations, AI coding assistants such as Cursor and GitHub Copilot generate vast amounts of usage telemetry through daily interactions, code generations, acceptances, and model invocations. While this data has the potential to reveal genuine adoption and proficiency, engineering leaders often struggle to interpret it effectively due to its volume, fragmentation across tools, and lack of unified analytical frameworks.

Traditional approaches rely on self-assessment surveys, manual spreadsheet tracking, or isolated admin dashboards that fail to connect telemetry with demonstrated competency or validated evidence of work. This leads to inefficiencies, inaccurate adoption reporting, and missed opportunities for targeted upskilling.

AI Adoption Analytics – Developer AI Proficiency Measurement System is developed in response to the growing demand for automated, accurate, and user-friendly AI adoption measurement. The system bridges the gap between raw tool metrics and actionable workforce intelligence by leveraging MySQL-based data warehousing, LLM-driven profile synthesis, vector-based competency matching, and pattern-based usage auditing. By presenting adoption information through profiles, scores, flags, and visual summaries, the platform empowers organizations to understand developer behaviour, improve planning, and make data-driven talent decisions.

1.2 Problem Definition

AI adoption data plays a critical role in evaluating engineering productivity, planning training programmes, identifying power users, and detecting misuse or superficial usage. However, in practical enterprise scenarios, this data is often scattered across Cursor Admin APIs, GitHub Copilot metrics, local evidence folders, and ad-hoc reports that are difficult to consolidate without proper analytical tooling.

Engineering managers frequently lack a single system that can: (a) sync multi-source telemetry into a relational store, (b) merge quantitative usage with qualitative evidence, (c) score engineers against a structured competency framework, and (d) flag suspicious usage patterns such as end-of-period activity stuffing or single-day dominance within a review window.

AI Adoption Analytics aims to solve these issues by providing a comprehensive data-driven platform capable of processing Cursor and Copilot inputs, performing detailed analysis, generating proficiency insights, and auditing weekly usage patterns. By automating the analytical workflow and presenting easy-to-understand outputs, the

system addresses the core problem of converting complex AI-tool data into meaningful, actionable knowledge.

1.3 Objectives of the Project

The primary objectives of the project are:

1. Ensure accurate, structured storage of Cursor and Copilot telemetry in MySQL through automated API sync.
2. Merge telemetry with uploaded evidence artifacts (PDF, DOCX, images) for richer developer context.
3. Generate LLM-based engineer profiles summarizing usage behaviour, model preferences, and work patterns.
4. Score profiles against a 43-item AI competency framework using vector similarity (Pinecone + BGE embeddings).
5. Detect suspicious usage patterns through a dedicated Auditor Agent analysing 7-day daily activity series.
6. Support batch processing of entire engineering rosters with parallel workers and MySQL result snapshots.
7. Provide an accessible Streamlit interface for single-user runs, batch execution, and audit visualization.
8. Enable evidence summarization and validation using multimodal LLMs for framework-aligned proof.

1.4 Scope of the Project

The scope encompasses the following areas:

Data Acquisition and Synchronization

- Cursor Admin API and Analytics API ingestion into MySQL.
- GitHub Copilot org metrics sync (seats, daily usage, breakdowns).
- Email linkage between GitHub logins and corporate identities.

Evidence Processing

- Summarization of files in per-user evidence folders.
- Validation of evidence against competency checklist items.

Profile and Scoring Pipeline

- Merged context building from telemetry + evidence + Copilot.
- Profile narrative generation via Hugging Face LLM or template fallback.
- Embedding and vector comparison against ingested competencies.

Usage Auditing

- 7-day window analysis for concentration, spikes, and end-of-period loading.

- Risk scoring and flag generation per user.

User Interface and Batch Operations

- Streamlit web application for pipeline orchestration.
- CLI batch pipeline with worker pools and MySQL snapshots.

1.5 Significance of the Project

9. Provides objective, data-driven visibility into how engineers use AI coding tools.
10. Connects telemetry with competency frameworks for structured talent assessment.
11. Reduces manual effort in adoption reporting through automated sync and batch pipelines.
12. Detects anomalous usage patterns that may indicate superficial or rushed adoption.
13. Demonstrates a practical multi-agent architecture applicable to enterprise AI governance.

CHAPTER 2 — RESEARCH METHODOLOGY

This chapter outlines the research methodology adopted for the development of AI Adoption Analytics – Developer AI Proficiency Measurement System. The methodology defines the systematic process followed to collect data, analyse patterns, apply AI techniques, validate outputs, and evaluate the system.

2.1 Research Approach

The research approach combines applied research, analytical research, and systems development.

Quantitative Research Approach

- Numerical telemetry: lines added, applies, accepts, tokens, costs, interactions.
- Measuring adoption intensity, active days, and competency match counts.
- Building risk scores and concentration metrics from daily activity series.

Data-Driven Analytical Approach

- Descriptive analytics: what usage occurred in the period?
- Diagnostic analytics: why did audit flags fire?
- Comparative analytics: how do users rank on telemetry scores?

AI and Machine Learning–Based Approach

- LLM-generated profiles from merged telemetry and evidence context.
- Embedding-based similarity for competency item matching.
- Multimodal evidence summarization for document and image artifacts.

Iterative and Incremental Approach

- Requirement analysis → prototype → testing → refinement cycles.
- Modular agents: sync, evaluator, validator, auditor, profile builder.

2.2 Data Collection and Requirements Analysis

The initial stage involved understanding enterprise AI-tool data characteristics and designing MySQL schemas to store them reliably.

2.1 Telemetry Data Characteristics

Cursor daily usage datasets include:

- User email and usage date
- Lines added/deleted and accepted lines
- Applies, accepts, rejects, tabs shown/accepted
- Most-used model and client version

- Active-day indicators

GitHub Copilot daily usage datasets include:

- GitHub login mapped to corporate email
- User-initiated interactions and code generation/acceptance counts
- Lines suggested and lines added
- Chat and agent usage flags
- Breakdowns by IDE, feature, language, and model

During analysis, several real-world issues were identified: missing days in sync windows, GitHub login-to-email mismatches, sparse evidence uploads, and multi-model usage requiring aggregation before profiling.

2.3 Research Design

2.3.1 Problem Identification

Examination of enterprise AI adoption workflows revealed:

- No unified view across Cursor and Copilot
- Manual profiling and subjective level assignment
- Difficulty auditing usage authenticity at scale
- Lack of stored batch results for historical comparison

2.3.2 Requirement Specification

Functional Requirements:

- Sync Cursor and Copilot APIs into MySQL
- Upload and summarize per-user evidence files
- Build merged context and generate profiles
- Score against 43 competency items
- Run 7-day usage-pattern audits
- Batch process full team rosters
- Web UI for pipeline execution and result viewing

Non-Functional Requirements:

- Scalability for hundreds of users via parallel workers
- Idempotent MySQL upserts and snapshot storage
- Configurable LLM provider with template fallback
- Modular, maintainable Python codebase

2.3.3 Conceptual Framework

- Ingestion Layer → Cursor/Copilot API sync into MySQL

- Evidence Layer → Summarize and validate uploaded artifacts
- Context Layer → Merge telemetry, evidence, and Copilot data
- Profile Layer → LLM/template profile generation
- Scoring Layer → Embedding + Pinecone competency comparison
- Audit Layer → 7-day pattern analysis and risk flags
- Presentation Layer → Streamlit UI and CLI batch tools
- Storage Layer → fact_user_proficiency_snapshot and sync tables

2.4 Development Methodology

A hybrid Agile methodology was adopted with iterative sprint cycles:

14. Planning: define module goals (sync, profile, audit)
15. Design: schema design, agent interfaces, pipeline flow
16. Implementation: Python modules, MySQL integration, Streamlit UI
17. Testing: single-user pilots, batch dry-runs, SQL validation
18. Review: refine thresholds, improve error handling, optimize workers

My contributions included:

- Implementing and testing the profile and audit pipeline components
- Integrating Cursor and Copilot data flows into unified context building
- Configuring competency ingestion and scoring workflows
- Running batch pipeline pilots and validating MySQL snapshot outputs
- Developing and testing Streamlit usage-audit visualizations

2.5 Evaluation Methodology

Evaluation involved multiple approaches:

Functional Evaluation: Each pipeline stage (sync, merge, profile, score, audit) verified end-to-end.

Data Quality Evaluation: Row counts, date coverage, and email linkage accuracy checked via SQL.

Scoring Evaluation: Telemetry scores, matched item counts, and suggested levels reviewed per user.

Audit Evaluation: Flag codes, risk scores, and daily activity charts validated against known patterns.

Performance Evaluation: Batch worker throughput and single-user pipeline latency measured.

UI Evaluation: Streamlit tabs tested for pipeline run, batch execution, and audit display.

2.6 Summary of Research Methodology

The methodology provided a systematic approach to developing a functional AI adoption measurement platform, ensuring thorough understanding of enterprise challenges, careful design, iterative development, detailed evaluation, and practical validation on real telemetry from R Systems International.

CHAPTER 3 — SYSTEM FLOW CHART

3.1 Overview of System Flow

AI Adoption Analytics follows a structured workflow consisting of seven major stages:

19. Data Input and API Synchronization
20. MySQL Storage and Email Linkage
21. Evidence Summarization and Validation
22. Context Merging and Profile Generation
23. Competency Embedding and Scoring
24. Usage Pattern Auditing
25. Output Delivery via Streamlit and Batch Snapshots

Each stage is interconnected, ensuring smooth data flow from raw API responses to stored proficiency snapshots and audit verdicts.

3.2 Detailed System Flow

Step 1 — Sync Initiation

`cursor_mysql_sync/main.py` pulls Cursor team members, daily usage, events, insights, and Copilot metrics for the selected date window. Results upsert into MySQL with `sync_run` tracking.

Step 2 — Evidence Processing

Files under `evidence_evaluator/evidences/<email>/` are summarized by `summarize_evidence.py` and optionally validated against competency items.

Step 3 — Context Merge

`build_merged_context.py` combines Cursor telemetry, Copilot MySQL data, and evidence summaries into a single context JSON.

Step 4 — Profile Creation

`create_profile.py` invokes Hugging Face LLM (Qwen-VL) or template mode to produce a narrative profile and summary.

Step 5 — Embedding and Scoring

`embed_profile.py` vectorizes the narrative; `compare_competencies.py` matches against 43 Pinecone-ingested items and computes `telemetry_score` and `suggested_level_v1`.

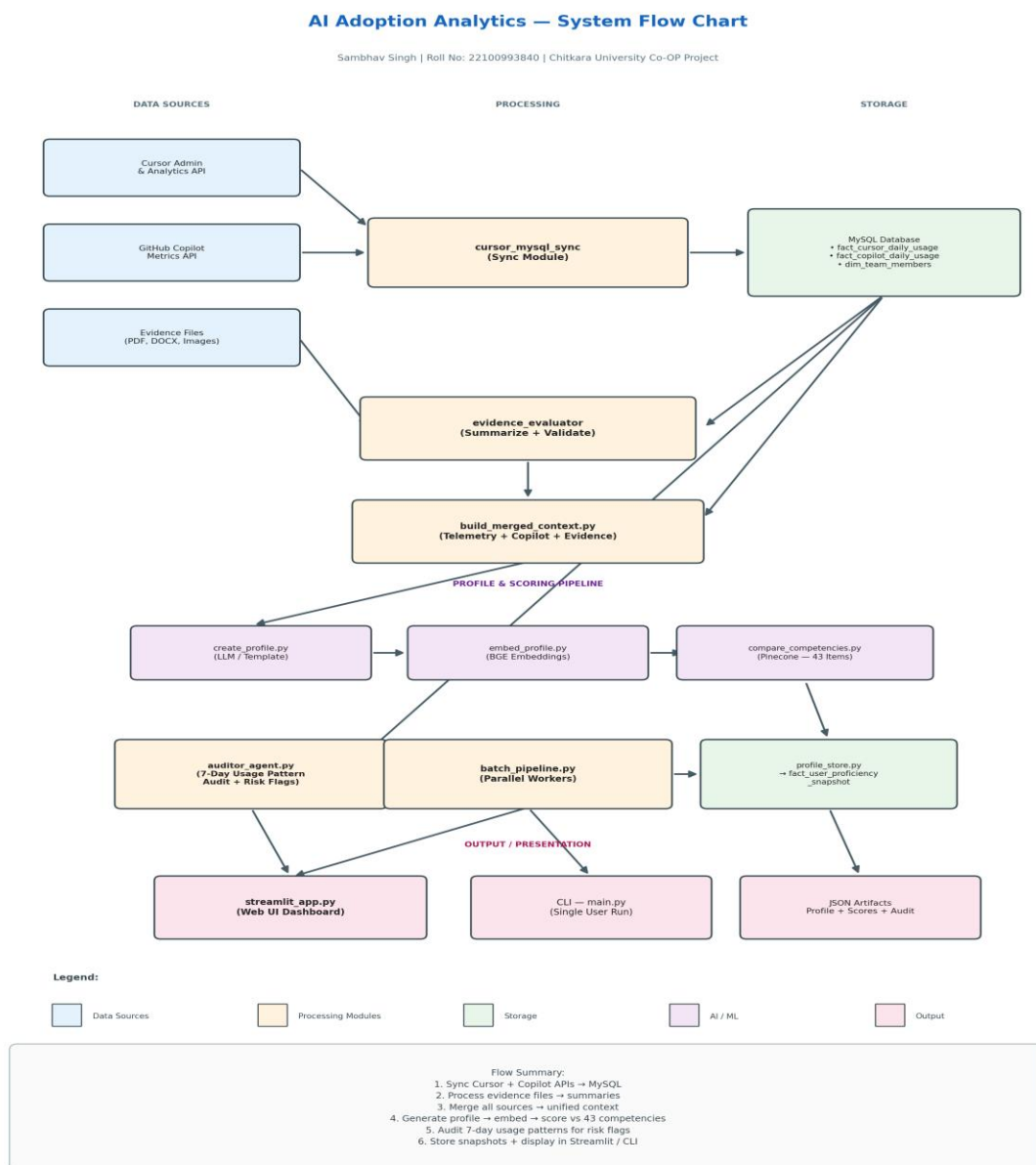
Step 6 — Usage Audit

auditor_agent.py reads fact_cursor_daily_usage for the 7-day window, computes activity weights, flags, risk score, and spike days.

Step 7 — Persistence and UI

profile_store.py upserts fact_user_proficiency_snapshot; Streamlit displays profiles, scores, and audit results; batch_pipeline.py runs all users in parallel.

System Flow (Textual Diagram):



CHAPTER 4 — TOOLS AND TECHNOLOGIES USED

4.1 Introduction to Technology Stack

The development of AI Adoption Analytics – Developer AI Proficiency Measurement System required modern data engineering tools, AI/LLM frameworks, vector databases, relational storage, and web UI technologies selected for scalability, modularity, and enterprise integration.

4.2 Programming Language — Python

Python was chosen as the primary language due to strong support for data processing, AI ecosystems, API clients, and rapid prototyping. All agents, sync modules, and the Streamlit UI are implemented in Python 3.

4.3 Database — MySQL

MySQL stores:

- dim_cursor_team_members, fact_cursor_daily_usage, fact_cursor_usage_events
- dim_copilot_users, fact_copilot_daily_usage, fact_copilot_breakdown
- fact_user_proficiency_snapshot, profile_batch_runs, sync_runs

4.4 Web Interface — Streamlit

Streamlit (streamlit_app.py) provides tabs for single-user pipeline execution, batch processing, step-by-step runs, results viewing, usage audit, evidence management, and admin functions.

4.5 AI and LLM Technologies

Hugging Face Inference API:

- Profile LLM: Qwen/Qwen3-VL-8B-Instruct (multimodal)
- Evidence summarization via HF_TOKEN

Embeddings:

- Local BAAI/bge-small-en-v1.5 (384 dimensions)
- Pinecone vector index: aiev-competencies-bge

4.6 External APIs

- Cursor Admin API and Analytics API
- GitHub Copilot org metrics API (users-1-day reports, billing seats)

4.7 Data Processing Libraries

- mysql-connector-python — database connectivity
- requests — HTTP API clients

- `python-dotenv` — environment configuration
- `concurrent.futures` — parallel batch workers

4.8 Development Tools

- `Git` — version control
- `pip` / `venv` — dependency management
- `PowerShell` / `CLI` — pipeline execution
- `JSON` — context, profile, score, and audit artifacts

CHAPTER 5 — SOLUTION IMPLEMENTATION

5.1 Implementation Approach

Implementation followed a modular, layered, multi-agent approach. Each component operates independently while sharing MySQL and JSON artifacts as integration points.

5.2 System Architecture

Layer 1 — Data Sync (cursor_mysql_sync/)

- main.py orchestrates Cursor and Copilot sync
- store_daily_usage.py, store_usage_events.py, store_insights.py
- copilot_sync.py for GitHub Copilot NDJSON reports
- link_copilot_emails.py for login-to-email mapping

Layer 2 — Evidence (evidence_evaluator/)

- summarize_evidence.py — multimodal file summarization
- validate_evidence.py — competency-aligned validation

Layer 3 — Profile Builder (build_user_profile/)

- fetch_user_data.py — Cursor telemetry from MySQL
- fetch_copilot_data.py — Copilot context from MySQL
- build_merged_context.py — unified context JSON
- create_profile.py — LLM/template profile generation
- embed_profile.py + compare/compare_competencies.py — scoring
- profile_store.py — MySQL snapshot persistence

Layer 4 — Auditor (auditor_agent.py)

- 7-day window fetch from fact_cursor_daily_usage
- Activity weighting, Gini concentration, spike detection
- Risk score and audit_status: ok | review | high_risk | no_data

Layer 5 — Orchestration

- main.py — single-user CLI commands
- batch_pipeline.py — parallel all-user processing
- streamlit_app.py — web UI

5.3 MySQL Schema Implementation

schema.sql defines Cursor and Copilot fact/dimension tables. schema_profile.sql adds profile_batch_runs and fact_user_proficiency_snapshot for batch result storage.

5.4 Profile Pipeline Implementation

The four-step pipeline executed per user:

26. Merge context (telemetry + evidence + Copilot)
27. Create profile (LLM narrative + summary)
28. Embed profile narrative vector
29. Compare against 43 competency items → scores JSON

5.5 Auditor Agent Implementation

Activity weights for Cursor daily rows:

- total_lines_added: weight 1.0
- total_applies: weight 3.0
- total_accepts: weight 3.0
- total_tabs_accepted: weight 1.0

Flag codes include: single_day_dominant, lumpy_usage, end_of_period_loaded, spike_day, single_active_day.

5.6 Batch Pipeline Implementation

batch_pipeline.py reads roster from dim_cursor_team_members, optionally filters users with events in the period (--require-events), runs pipeline_runner per user with ThreadPoolExecutor workers, and upserts fact_user_proficiency_snapshot.

5.7 Streamlit UI Implementation

Key UI capabilities:

- Sidebar: user email, period selection, template-only mode
- Run full pipeline or usage-audit-only
- Batch tab: dry-run, workers, require-events flags
- Audit tab: risk metrics, daily chart, flag list, JSON download
- Results tab: profile and score display

5.8 Edge Case Handling

- Missing evidence folder → telemetry-only profile
- Copilot tables absent → graceful skip in context merge
- LLM unavailable → template profile via --no-llm
- Sparse usage → audit_status no_data with zero risk
- Per-user pipeline errors → captured in pipeline_status and error_message

5.9 My Contributions Summary

- Profile and scoring pipeline integration and testing
- Copilot data fetch and merged context building
- Auditor agent testing and Streamlit audit visualization
- Batch pipeline execution and MySQL snapshot validation.

CHAPTER 6 — MAJOR FINDINGS / OUTCOMES / OUTPUT / RESULTS

6.1 Introduction to Results

The AI Adoption Analytics system was tested with real enterprise telemetry and evidence from R Systems International engineering users. Results demonstrate successful operation across sync, profiling, scoring, auditing, and batch processing.

6.2 Pipeline Execution Results

End-to-end pipeline stages executed successfully:

- Cursor MySQL sync completed with daily usage and event rows
- Merged context JSON produced per user per period
- Profile JSON and scores JSON written to build_user_profile/output/<email>/
- MySQL snapshots upserted via profile_store.py
- Auditor agent returned structured JSON with metrics and flags

6.3 Sample User Results — Sambhav Singh

Period: 2026-05-19 to 2026-05-25 (test run on project developer account)

Metric	Value
User email	sambhav.singh@rsystems.com
Telemetry score	62.8
Matched competencies	27 / 43
Suggested level	Below Aspirant
Active days (Cursor)	2
Lines added	1,808
Lines accepted	1,519
Total API events	60

6.4 Model Usage Distribution

Cursor model usage for the sample period:

- claude-opus-4-7-thinking-xhigh: 16 events, ~9.52 USD cost
- claude-4.6-sonnet-medium-thinking: 19 events, ~3.75 USD cost
- default: 25 events, ~1.15 USD cost
- Total tokens: ~11.2M; total cost: ~14.42 USD

6.5 Work Category Insights

Cursor insight distribution showed diverse work types:

- Configuration — 22.2%
- Code Refactoring — 22.2%
- Code Explanation — 22.2%
- New Features — 11.1%
- Learning — 11.1%
- Bug Fixing & Debugging — 11.1%

6.6 Competency Scoring Results

Vector similarity matching against 43 framework items produced `checked_item_ids`, competency rollups, and `suggested_level_v1`. The match threshold default is 0.68. Scores are provisional telemetry-based matches, not official certification.

6.7 Auditor Agent Results

The 7-day auditor evaluates:

- `top1_share` — single-day dominance (threshold $\geq 60\%$)
- `top2_share` — lumpy usage (threshold $\geq 80\%$)
- `last2_share` — end-of-period loading (threshold $\geq 75\%$)
- `spike_day` — days $\geq 2\times$ median activity
- `concentration_gini` — activity spread across the week

Audit statuses assigned: `ok`, `review`, `high_risk`, `no_data`, or `error`. Risk scores range from 0.0 to 1.0 based on flag severity weights.

6.8 Batch Pipeline Results

Batch execution with `--workers 4` and `--require-events` processed multiple team members in parallel. Results queryable via:

```
SELECT user_email, suggested_level_v1, telemetry_score, matched_count,
pipeline_status FROM fact_user_proficiency_snapshot WHERE period_start = '2026-05-19' AND period_end = '2026-05-25';
```

6.9 Streamlit UI Results

The web interface successfully demonstrated:

- Single-user full pipeline execution from sidebar settings
- Usage audit tab with risk score, flags, and daily activity table
- Profile download and JSON export
- Batch dry-run roster preview

6.10 System Performance

- Single-user pipeline: completes in minutes depending on LLM latency
- Batch with 4 workers: parallel speedup vs sequential execution

- MySQL upserts: idempotent ON DUPLICATE KEY UPDATE
- Auditor: lightweight 7-row-per-user query pattern

6.11 Overall Outcomes

- 30. Unified multi-source AI adoption measurement platform
- 31. Automated proficiency scoring against competency framework
- 32. Usage-pattern auditing for adoption authenticity
- 33. Batch scalability for enterprise rosters
- 34. Streamlit and CLI interfaces for operators and analysts

6.12 Summary

This chapter presented major outcomes demonstrating robust performance across data synchronization, profile generation, competency scoring, usage auditing, and batch operations on real enterprise AI-tool telemetry.

CHAPTER 7 — CONCLUSION AND FUTURE SCOPE

7.1 Conclusion

The AI Adoption Analytics project successfully demonstrates the power of combining data engineering, large language models, vector search, and rule-based auditing to measure AI coding tool adoption in enterprise environments.

By integrating Cursor telemetry, GitHub Copilot metrics, evidence artifacts, and a structured competency framework, the system delivers a comprehensive platform for profiling, scoring, and auditing developer AI usage.

The profile and scoring modules transform raw telemetry into narrative profiles and quantitative proficiency levels. The auditor agent adds a governance layer by detecting concentrated or suspicious usage patterns within review windows.

The Streamlit interface and batch pipeline make the solution practical for operators managing large engineering organizations, storing historical snapshots in MySQL for trend analysis.

Overall, the project meets its core objectives and establishes a foundation for enterprise AI adoption intelligence at scale.

7.2 Limitations

- Scoring is provisional telemetry similarity, not official competency certification.
- Auditor currently focuses on Cursor daily usage; full Copilot audit integration is in progress.
- LLM profile quality depends on model availability and evidence completeness.
- Real-time streaming sync is not implemented; batch windows are calendar-day based.

7.3 Future Scope

Combined Cursor + Copilot Auditor: Extend `auditor_agent` to analyse `fact_copilot_daily_usage` and persist to `fact_usage_audit_snapshot`.

Organization Dashboard: Executive dashboards for adoption trends, level distribution, and audit flag rates.

Real-Time Sync: Scheduled cron jobs for daily automatic sync and snapshot refresh.

RAG over Evidence: Retrieval-augmented generation for richer evidence-to-competency linking.

Official Framework Integration: Workflow for human reviewer approval of matched competencies.

Predictive Models: Forecast adoption trajectories and training needs from historical snapshots.

Multi-Tool Expansion: Support additional AI assistants beyond Cursor and Copilot.

7.4 Final Summary

The AI Adoption Analytics project marks a significant step toward objective, scalable measurement of AI tool adoption in software engineering organizations. Its modular architecture, multi-agent design, and enterprise data integration make it a strong foundation for future AI workforce analytics innovations.

REFERENCES

1. Official Python documentation for data processing, automation, and system development used in pipeline module design.
2. MySQL documentation for relational schema design, upsert patterns, and query optimization.
3. Cursor Admin API and Analytics API documentation for team member and usage event ingestion.
4. GitHub Copilot metrics API documentation (users-1-day reports, billing seats) for org-level usage sync.
5. Hugging Face Inference API documentation for multimodal LLM profile and evidence summarization.
6. Pinecone vector database documentation for competency index management and similarity search.
7. BAAI/bge-small-en-v1.5 embedding model documentation for local vector generation.
8. Streamlit documentation for interactive web application development.
9. Research materials on AI adoption measurement, developer productivity, and competency frameworks.
10. Industry best practices for data pipeline design, batch processing, and observability.
11. Conceptual resources on vector similarity, embedding-based retrieval, and LLM prompt engineering.
12. Software engineering references on modular architecture, API-driven design, and Agile iteration.
13. General resources on anomaly detection, concentration metrics, and time-series pattern analysis.
14. Enterprise AI governance guidelines for responsible telemetry collection and usage auditing.
15. Open-source community tutorials on Python MySQL integration and parallel worker pools.

APPENDICES

Appendix A: MySQL Table Structure (Key Tables)

- fact_cursor_daily_usage — per-user per-day Cursor metrics
- fact_copilot_daily_usage — per-user per-day Copilot metrics
- fact_user_proficiency_snapshot — batch profile and score results
- dim_cursor_team_members — engineering roster
- dim_copilot_users — GitHub login to email mapping

Appendix B: CLI Commands

- python cursor_mysql_sync/main.py --days 30
- python main.py ingest
- python main.py all --user <email> --days 30
- python batch_pipeline.py --days 7 --workers 4 --require-events
- python auditor_agent.py --user <email> --days 7
- streamlit run streamlit_app.py

Appendix C: Sample Audit Output Fields

- user_email, period_start, period_end
- audit_status, risk_score
- flags: code, severity, detail
- metrics: total_activity, active_days, top1_share, concentration_gini
- daily: date, activity, is_active
- spike_days: list of ISO dates

Appendix D: Competency Framework

The system scores against 43 competency items ingested into Pinecone. Items cover AI-assisted development practices including prompting, debugging, refactoring, testing, and responsible AI usage. match_threshold default: 0.68.

Appendix E: Test Scenarios

- Single user with full telemetry and evidence
- Telemetry-only user (no evidence folder)
- Sparse usage (1–2 active days in window)
- Batch dry-run roster preview
- Template profile mode (--no-llm)
- Copilot tables missing graceful degradation
- Audit with --all and parallel workers